

Testing unitario – JUnit

Programación III - UNGS

Testing unitario

- Llamamos **testing unitario** (*unit testing*) a las actividades tendientes a testear **automáticamente** unidades individuales de código fuente, para determinar si funcionan correctamente.

Testing unitario

- Llamamos **testing unitario** (*unit testing*) a las actividades tendientes a testear **automáticamente** unidades individuales de código fuente, para determinar si funcionan correctamente.
 1. Como toda actividad de testing, el testing unitario no puede **garantizar** que no haya errores, pero proporciona cierto nivel de seguridad sobre el código.

Testing unitario

- Llamamos **testing unitario** (*unit testing*) a las actividades tendientes a testear **automáticamente** unidades individuales de código fuente, para determinar si funcionan correctamente.
 1. Como toda actividad de testing, el testing unitario no puede **garantizar** que no haya errores, pero proporciona cierto nivel de seguridad sobre el código.
 2. El testing unitario permite encontrar muchos errores antes de que el código se integre al resto del sistema.

Testing unitario

- Llamamos **testing unitario** (*unit testing*) a las actividades tendientes a testear **automáticamente** unidades individuales de código fuente, para determinar si funcionan correctamente.
 1. Como toda actividad de testing, el testing unitario no puede **garantizar** que no haya errores, pero proporciona cierto nivel de seguridad sobre el código.
 2. El testing unitario permite encontrar muchos errores antes de que el código se integre al resto del sistema.
- En un lenguaje orientado a objetos, por **unidad individual** nos referimos habitualmente a las clases y a sus métodos.

Testing unitario

- Llamamos **testing unitario** (*unit testing*) a las actividades tendientes a testear **automáticamente** unidades individuales de código fuente, para determinar si funcionan correctamente.
 1. Como toda actividad de testing, el testing unitario no puede **garantizar** que no haya errores, pero proporciona cierto nivel de seguridad sobre el código.
 2. El testing unitario permite encontrar muchos errores antes de que el código se integre al resto del sistema.
- En un lenguaje orientado a objetos, por **unidad individual** nos referimos habitualmente a las clases y a sus métodos.
- Los tests unitarios son creados por los programadores al momento de escribir el código de negocio.

JUnit

- Existen varios paquetes de testing unitario para cada entorno de desarrollo. En Java, el más popular es **JUnit**, desarrollado por Kent Beck, Erich Gamma, David Saff y Mike Clark, de la Universidad de Calgary (Canadá).

JUnit

- Existen varios paquetes de testing unitario para cada entorno de desarrollo. En Java, el más popular es **JUnit**, desarrollado por Kent Beck, Erich Gamma, David Saff y Mike Clark, de la Universidad de Calgary (Canadá).
- Para instalarlo, se debe incluir en el *build path* el archivo **junit.jar** (se realiza automáticamente en eclipse).

JUnit

- Existen varios paquetes de testing unitario para cada entorno de desarrollo. En Java, el más popular es **JUnit**, desarrollado por Kent Beck, Erich Gamma, David Saff y Mike Clark, de la Universidad de Calgary (Canadá).
- Para instalarlo, se debe incluir en el *build path* el archivo **junit.jar** (se realiza automáticamente en eclipse).
- Cada vez que se escribe una clase, por convención se agregan una o más clases con nombres terminados en "Test", que contienen los tests unitarios para la clase de negocio.

Ejemplo

- Por ejemplo, tenemos la clase Persona con su nombre y edad:

Ejemplo

- Por ejemplo, tenemos la clase Persona con su nombre y edad:

```

1  class Persona
2  {
3      private String nombre;
4      private int edad;
5
6      public Persona(String n, int e)
7      {
8          nombre = n;
9          edad = e;
10     }
11
12     public void cumplirAnios()
13     {
14         ++edad;
15     }

```

Ejemplo

- Escribimos un test unitario para testear `cumplirAnios()`:

Ejemplo

- Escribimos un test unitario para testear `cumplirAnios()`:

```

1  class PersonaTest
2  {
3      @Test
4      public void cumplirAniosTest()
5      {
6          Persona p = new Persona("Pepe", 17);
7          p.cumplirAnios();
8          assertEquals(18, p.getEdad());
9      }
10 }
```

Ejemplo

- Escribimos un test unitario para testear `cumplirAnios()`:

```

1  class PersonaTest
2  {
3      @Test
4      public void cumplirAniosTest()
5      {
6          Persona p = new Persona("Pepe", 17);
7          p.cumplirAnios();
8          assertEquals(18, p.getEdad());
9      }
10 }
```

- Este **caso de test** especifica que una persona creada con 17 años debe tener 18 años luego de ejecutar el método `cumplirAnios()` sobre ella.

Ejemplo

- Veamos la **estructura** del caso de test:

Ejemplo

- Veamos la **estructura** del caso de test:

```

1  // Setup de los datos (el "test fixture")
2  Persona p = new Persona(" Pepe", 17);
3
4  // Ejecutamos (exercise) la funcionalidad a testear
5  p.cumplirAnios();
6
7  // Verificamos que el resultado sea el esperado
8  assertEquals(18, p.getEdad());
  
```

Ejemplo

- Veamos la **estructura** del caso de test:

```

1  // Setup de los datos (el "test fixture")
2  Persona p = new Persona(" Pepe", 17);
3
4  // Ejecutamos (exercise) la funcionalidad a testear
5  p.cumplirAnios();
6
7  // Verificamos que el resultado sea el esperado
8  assertEquals(18, p.getEdad());

```

- Todos los tests deben seguir la estructura **setup-exercise-verify**, idealmente con una línea de código en cada una.

Ejemplo

- Veamos la **estructura** del caso de test:

```

1  // Setup de los datos (el "test fixture")
2  Persona p = new Persona(" Pepe", 17);
3
4  // Ejecutamos (exercise) la funcionalidad a testear
5  p.cumplirAnios();
6
7  // Verificamos que el resultado sea el esperado
8  assertEquals(18, p.getEdad());
  
```

- Todos los tests deben seguir la estructura **setup-exercise-verify**, idealmente con una línea de código en cada una.
- Puede haber también una cuarta etapa de **teardown** del test fixture.

Ejemplo

- Para ejecutar los tests unitarios, dentro de eclipse seleccionamos la acción **Run As** sobre el proyecto y seleccionamos la opción “JUnit Test”.

Ejemplo

- Para ejecutar los tests unitarios, dentro de eclipse seleccionamos la acción **Run As** sobre el proyecto y seleccionamos la opción “JUnit Test”.
- Se ejecutan todos los métodos calificados con @Test, y se informa el resultado de cada uno. Si hay tests que fallan, se informa cuáles fallaron y se puede acceder a su código.

Ejemplo

- Para ejecutar los tests unitarios, dentro de eclipse seleccionamos la acción **Run As** sobre el proyecto y seleccionamos la opción “JUnit Test”.
- Se ejecutan todos los métodos calificados con @Test, y se informa el resultado de cada uno. Si hay tests que fallan, se informa cuáles fallaron y se puede acceder a su código.
- Es buena práctica ejecutar frecuentemente todos los tests unitarios.

Ejemplo

- Para ejecutar los tests unitarios, dentro de eclipse seleccionamos la acción **Run As** sobre el proyecto y seleccionamos la opción “JUnit Test”.
- Se ejecutan todos los métodos calificados con @Test, y se informa el resultado de cada uno. Si hay tests que fallan, se informa cuáles fallaron y se puede acceder a su código.
- Es buena práctica ejecutar frecuentemente todos los tests unitarios.
 1. Idealmente, todos los tests unitarios se deben ejecutar cada vez que se hace un cambio en la funcionalidad.

Ejemplo

- Para ejecutar los tests unitarios, dentro de eclipse seleccionamos la acción **Run As** sobre el proyecto y seleccionamos la opción “JUnit Test”.
- Se ejecutan todos los métodos calificados con @Test, y se informa el resultado de cada uno. Si hay tests que fallan, se informa cuáles fallaron y se puede acceder a su código.
- Es buena práctica ejecutar frecuentemente todos los tests unitarios.
 1. Idealmente, todos los tests unitarios se deben ejecutar cada vez que se hace un cambio en la funcionalidad.
 2. Si esto no es posible, se deben ejecutar completos cada vez que se hace **commit** del código al repositorio. Si hay errores, se debe corregir inmediatamente.

Las *assertions*

- Se utilizan **assertions** para testear condiciones que se deben cumplir durante la ejecución de los tests unitarios.

Las *assertions*

- Se utilizan **assertions** para testear condiciones que se deben cumplir durante la ejecución de los tests unitarios.
 1. **assertEquals(expected, actual)**
assertArrayEquals(expected, actual)
Requiere que los dos parámetros coincidan.

Las *assertions*

- Se utilizan **assertions** para testear condiciones que se deben cumplir durante la ejecución de los tests unitarios.
 1. **assertEquals(expected, actual)**
assertArrayEquals(expected, actual)
Requiere que los dos parámetros coincidan.
 2. **assertTrue(cond)**
Requiere que la condición sea verdadera.

Las *assertions*

- Se utilizan **assertions** para testear condiciones que se deben cumplir durante la ejecución de los tests unitarios.
 1. **assertEquals(expected, actual)**
assertArrayEquals(expected, actual)
Requiere que los dos parámetros coincidan.
 2. **assertTrue(cond)**
Requiere que la condición sea verdadera.
 3. **assertFalse(cond)**
Requiere que la condición sea falsa.

Las *assertions*

- Se utilizan **assertions** para testear condiciones que se deben cumplir durante la ejecución de los tests unitarios.
 1. **assertEquals(expected, actual)**
assertArrayEquals(expected, actual)
Requiere que los dos parámetros coincidan.
 2. **assertTrue(cond)**
Requiere que la condición sea verdadera.
 3. **assertFalse(cond)**
Requiere que la condición sea falsa.
 4. **assertNull(obj)**
Requiere que la referencia sea **null**.

Las *assertions*

- Se utilizan **assertions** para testear condiciones que se deben cumplir durante la ejecución de los tests unitarios.
 1. **assertEquals(expected, actual)**
assertArrayEquals(expected, actual)
 Requiere que los dos parámetros coincidan.
 2. **assertTrue(cond)**
 Requiere que la condición sea verdadera.
 3. **assertFalse(cond)**
 Requiere que la condición sea falsa.
 4. **assertNull(obj)**
 Requiere que la referencia sea **null**.
 5. **assertNotNull(obj)**
 Requiere que la referencia no sea **null**.

Las *annotations*

- Se puede escribir código que se ejecuta antes de realizar los tests, por ejemplo para inicializar elementos comunes a los tests.

Las *annotations*

- Se puede escribir código que se ejecuta antes de realizar los tests, por ejemplo para inicializar elementos comunes a los tests.

```

1  class MateriaTest
2  {
3      private List<Alumno> alumnos;
4
5      @Before
6      public void inicializar()
7      {
8          alumnos = new List<Alumno>();
9          alumnos.add(new Alumno(" Jose Perez", " 32514521/2011"));
10
11         alumnos.add(new Alumno(" Josefina Lopez", " 34521651/2011"));
12         ...
13     }
14     ...

```

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:
 1. **@Test**: Identifica a un método de test.

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:
 1. **@Test:** Identifica a un método de test.
 2. **@Before:** Se ejecuta este método antes de cada test, habitualmente para inicializar los tests o establecer una conexión con la base de datos.

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:
 1. **@Test:** Identifica a un método de test.
 2. **@Before:** Se ejecuta este método antes de cada test, habitualmente para inicializar los tests o establecer una conexión con la base de datos.
 3. **@After:** Se ejecuta este método después de los tests, habitualmente para cerrar conexiones o restaurar los valores originales, si corresponde.

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:
 1. **@Test:** Identifica a un método de test.
 2. **@Before:** Se ejecuta este método antes de cada test, habitualmente para inicializar los tests o establecer una conexión con la base de datos.
 3. **@After:** Se ejecuta este método después de los tests, habitualmente para cerrar conexiones o restaurar los valores originales, si corresponde.
 4. **@Ignore:** No ejecuta el método de test. Se utiliza para omitir tests que pueden llevar demasiado tiempo.

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:
 1. **@Test:** Identifica a un método de test.
 2. **@Before:** Se ejecuta este método antes de cada test, habitualmente para inicializar los tests o establecer una conexión con la base de datos.
 3. **@After:** Se ejecuta este método después de los tests, habitualmente para cerrar conexiones o restaurar los valores originales, si corresponde.
 4. **@Ignore:** No ejecuta el método de test. Se utiliza para omitir tests que pueden llevar demasiado tiempo.
 5. **@Test(expected = Exception.class):** Falla si el método no lanza la excepción especificada.

Las *annotations*

- Se pueden aplicar las siguientes **anotaciones** (*annotations*) a los métodos de la clase de test:
 1. **@Test**: Identifica a un método de test.
 2. **@Before**: Se ejecuta este método antes de cada test, habitualmente para inicializar los tests o establecer una conexión con la base de datos.
 3. **@After**: Se ejecuta este método después de los tests, habitualmente para cerrar conexiones o restaurar los valores originales, si corresponde.
 4. **@Ignore**: No ejecuta el método de test. Se utiliza para omitir tests que pueden llevar demasiado tiempo.
 5. **@Test(expected = Exception.class)**: Falla si el método no lanza la excepción especificada.
 6. **@Test(timeout = 100)**: Falla si el método no se ejecuta dentro del límite de tiempo especificado (en milisegundos).

Beneficios del testing unitario

1. Permite tener cierta seguridad en cuanto a la **correcta implementación** de las unidades que se testean.

Beneficios del testing unitario

1. Permite tener cierta seguridad en cuanto a la **correcta implementación** de las unidades que se testean.
 - Si se testea toda la funcionalidad relevante, entonces es posible tener cierto grado de certeza de que la implementación es correcta.

Beneficios del testing unitario

1. Permite tener cierta seguridad en cuanto a la **correcta implementación** de las unidades que se testean.
 - Si se testea toda la funcionalidad relevante, entonces es posible tener cierto grado de certeza de que la implementación es correcta.
 - Los tests proporcionan un entorno aislado para testear cada parte de la funcionalidad del sistema.

Beneficios del testing unitario

1. Permite tener cierta seguridad en cuanto a la **correcta implementación** de las unidades que se testean.
 - Si se testea toda la funcionalidad relevante, entonces es posible tener cierto grado de certeza de que la implementación es correcta.
 - Los tests proporcionan un entorno aislado para testear cada parte de la funcionalidad del sistema.
 - Es importante que cada test apunte a una parte distinta de la funcionalidad. Así, es fácil **aislar un problema** en función de los tests que fallan.

Beneficios del testing unitario

1. Permite tener cierta seguridad en cuanto a la **correcta implementación** de las unidades que se testean.
 - Si se testea toda la funcionalidad relevante, entonces es posible tener cierto grado de certeza de que la implementación es correcta.
 - Los tests proporcionan un entorno aislado para testear cada parte de la funcionalidad del sistema.
 - Es importante que cada test apunte a una parte distinta de la funcionalidad. Así, es fácil **aislar un problema** en función de los tests que fallan.
 - Los tests no tienen que ser repetitivos, para no tener que modificar demasiados tests si cambia la interfaz o la funcionalidad.

Beneficios del testing unitario

2. El código de test bien diseñado proporciona **documentación** del código de producción!

Beneficios del testing unitario

2. El código de test bien diseñado proporciona **documentación** del código de producción!

Well designed tests are small isolated snippets of code that call into the system being tested, expecting certain results. A programmer can read the tests to understand what the tested code is supposed to do. So the tests are documents. They are written in a language you understand. They are utterly unambiguous. They are so formal that they execute. And they cannot get out of sync with the application.

That's pretty close to documentation nirvana. I've certainly seen my share of documents that were hard to read, ambiguous, informal, and out of sync with the application. That fact that a good suite of tests cures those ills makes the tests pretty important. – Robert Martin, 2014.

Beneficios del testing unitario

3. Al diseñar el código para ser testeado, la calidad del **diseño** mejora sustancialmente.

Beneficios del testing unitario

3. Al diseñar el código para ser testeado, la calidad del **diseño** mejora sustancialmente.

Well designed tests force a certain degree of decoupling. Often that decoupling is beneficial to the design of the system (...) No one can doubt that well-considered decoupling is beneficial. Tests provide an opportunity for that consideration; and that adds to the importance of the tests. – Robert Martin, 2014.

Beneficios del testing unitario

4. Una suite de test completa permite ganar **confianza** en el código, permitiendo refactorizaciones frecuentes.

Beneficios del testing unitario

4. Una suite de test completa permite ganar **confianza** en el código, permitiendo refactorizaciones frecuentes.

A well designed test suite with a high degree of coverage eliminates, or at least strongly mitigates the fear of change. And when you aren't afraid to change your code, you will clean it. And if you clean it, it won't rot. And if it doesn't rot, then the software team can go fast.

(...) The vast majority of programmers say that they have indeed been significantly slowed down by bad code. I mean, honestly, who hasn't? So it stands to reason that if we keep the code clean, we won't be slowed down by bad code. And that means a suite of tests is a key to going fast.

Let me state that more strongly. If you have a suite of tests that you can execute quickly, and if you trust that suite of tests enough, then you will not be afraid to change the code. That makes the code flexible. – Robert Martin, 2014.

Limitaciones del testing unitario

1. El testing nunca garantiza la **ausencia de errores**!

Limitaciones del testing unitario

1. El testing nunca garantiza la **ausencia de errores!**

Testing shows the presence, not the absence of bugs.
– Edsger Dijkstra, 1969.

Limitaciones del testing unitario

1. El testing nunca garantiza la **ausencia de errores**!

Testing shows the presence, not the absence of bugs.
– Edsger Dijkstra, 1969.

2. Requiere de **revisión permanente**, para evitar que los tests queden desactualizados o resulten insuficientes a medida que la funcionalidad del sistema aumenta.

Limitaciones del testing unitario

1. El testing nunca garantiza la **ausencia de errores**!

Testing shows the presence, not the absence of bugs.
– Edsger Dijkstra, 1969.

2. Requiere de **revisión permanente**, para evitar que los tests queden desactualizados o resulten insuficientes a medida que la funcionalidad del sistema aumenta.
3. Puede ser complicado/costoso definir tests **ortogonales** y desacoplados entre sí, de modo tal que cada caso de test apunte a una funcionalidad específica y distinta del resto.

Limitaciones del testing unitario

1. El testing nunca garantiza la **ausencia de errores**!

Testing shows the presence, not the absence of bugs.
– Edsger Dijkstra, 1969.

2. Requiere de **revisión permanente**, para evitar que los tests queden desactualizados o resulten insuficientes a medida que la funcionalidad del sistema aumenta.
3. Puede ser complicado/costoso definir tests **ortogonales** y desacoplados entre sí, de modo tal que cada caso de test apunte a una funcionalidad específica y distinta del resto.
4. El testing unitario no permite encontrar errores derivados de la **integración** entre módulos, dado que cada módulo se testea por separado.

Ejemplo

- Test unitarios para una clase que representa una materia, en el contexto de un sistema de registro de inscripciones a materias.

Ejemplo

- Test unitarios para una clase que representa una materia, en el contexto de un sistema de registro de inscripciones a materias.

```

1  class MateriaTest
2  {
3      @Test
4      public void preInscripcionTest()
5      {
6          Materia m = new Materia("Programacion III");
7          Alumno a = new Alumno("Jose Perez", "32514521/2011");
8          assertFalse(m.estaInscripto(a));
9      }
10     ...

```

Ejemplo

- Test unitarios para una clase que representa una materia, en el contexto de un sistema de registro de inscripciones a materias.

```

1  @Test
2  public void postInscripcionTest()
3  {
4      Materia m = new Materia("Programacion III");
5      Alumno a = new Alumno("Jose Perez", "32514521/2011");
6      m.inscribir(a);
7      assertTrue(m.estaInscripto(a));
8  }
    
```

Ejemplo

- Es importante testear los **casos de borde**.

Ejemplo

- Es importante testear los **casos de borde**.

```

1      @Test
2      public void sinInscriptosTest()
3      {
4          Materia m = new Materia("Programacion III");
5          assertEquals(0, m.cantidadAprobados());
6      }
    
```

Ejemplo

- Es importante testear los **casos de borde**.

```

1      @Test
2      public void unInscriptoTest()
3      {
4          Materia m = new Materia("Programacion III");
5          m.inscribir(new Alumno("Jose Perez", "32514521/2011"));
6          m.setNotaFinal(a, 7);
7
8          assertEquals(7, m.promedioAprobados());
9      }
    
```

Técnicas de testing

- Para el diseño de los tests ...

Técnicas de testing

- Para el diseño de los tests ...
 1. Testear las **excepciones** que lanza el método, idealmente especificadas en su declaración. Las excepciones forman parte de la interfaz pública de un método!

Técnicas de testing

- Para el diseño de los tests ...
 1. Testear las **excepciones** que lanza el método, idealmente especificadas en su declaración. Las excepciones forman parte de la interfaz pública de un método!
 2. Testear el **happy path**, con datos que hacen que el método se comporte de la forma esperada.

Técnicas de testing

- Para el diseño de los tests ...
 1. Testear las **excepciones** que lanza el método, idealmente especificadas en su declaración. Las excepciones forman parte de la interfaz pública de un método!
 2. Testear el **happy path**, con datos que hacen que el método se comporte de la forma esperada.
 3. Testear los **casos de borde**, tanto de la especificación como del código (arreglos/strings vacíos, elementos al inicio/final del arreglo, etc.).

Técnicas de testing

- Para el diseño de los tests ...
 1. Testear las **excepciones** que lanza el método, idealmente especificadas en su declaración. Las excepciones forman parte de la interfaz pública de un método!
 2. Testear el **happy path**, con datos que hacen que el método se comporte de la forma esperada.
 3. Testear los **casos de borde**, tanto de la especificación como del código (arreglos/strings vacíos, elementos al inicio/final del arreglo, etc.).
- No se deben **repetir tests**, dado que en ese caso es más trabajo arreglar los tests cuando cambia la funcionalidad.

Técnicas de testing

- **Test First Development.** Escribir los casos de test **antes** de implementar cada método. El objetivo es pensar **qué** se debe hacer antes de pensar **cómo** se implementará el método en cuestión.

Técnicas de testing

- **Test First Development.** Escribir los casos de test **antes** de implementar cada método. El objetivo es pensar **qué** se debe hacer antes de pensar **cómo** se implementará el método en cuestión.
- **Test-Driven Development (TDD).** Escribir cada **caso de test individual antes** de su implementación, y a continuación implementarlo.

Técnicas de testing

- **Test First Development.** Escribir los casos de test **antes** de implementar cada método. El objetivo es pensar **qué** se debe hacer antes de pensar **cómo** se implementará el método en cuestión.
- **Test-Driven Development (TDD).** Escribir cada **caso de test individual antes** de su implementación, y a continuación implementarlo.

Las tres leyes de TDD:

Técnicas de testing

- **Test First Development.** Escribir los casos de test **antes** de implementar cada método. El objetivo es pensar **qué** se debe hacer antes de pensar **cómo** se implementará el método en cuestión.
- **Test-Driven Development (TDD).** Escribir cada **caso de test individual antes** de su implementación, y a continuación implementarlo.

Las tres leyes de TDD:

1. No se debe escribir código de producción sin tener antes un test que falla para ese código.

Técnicas de testing

- **Test First Development.** Escribir los casos de test **antes** de implementar cada método. El objetivo es pensar **qué** se debe hacer antes de pensar **cómo** se implementará el método en cuestión.
- **Test-Driven Development (TDD).** Escribir cada **caso de test individual antes** de su implementación, y a continuación implementarlo.

Las tres leyes de TDD:

1. No se debe escribir código de producción sin tener antes un test que falla para ese código.
2. No se debe escribir más que un test unitario que falla.

Técnicas de testing

- **Test First Development.** Escribir los casos de test **antes** de implementar cada método. El objetivo es pensar **qué** se debe hacer antes de pensar **cómo** se implementará el método en cuestión.
- **Test-Driven Development (TDD).** Escribir cada **caso de test individual antes** de su implementación, y a continuación implementarlo.

Las tres leyes de TDD:

1. No se debe escribir código de producción sin tener antes un test que falla para ese código.
2. No se debe escribir más que un test unitario que falla.
3. No se debe escribir más código de producción que el estrictamente necesario para que pase el test que falla.

Consejos

1. Visualizar el testing unitario como una **disciplina** asociada con las actividades de desarrollo de software.

Consejos

1. Visualizar el testing unitario como una **disciplina** asociada con las actividades de desarrollo de software.
2. Escribir un caso de test para cada **elemento individual de funcionalidad**. Cada afirmación individual en la especificación debe tener al menos un caso de test.

Consejos

1. Visualizar el testing unitario como una **disciplina** asociada con las actividades de desarrollo de software.
2. Escribir un caso de test para cada **elemento individual de funcionalidad**. Cada afirmación individual en la especificación debe tener al menos un caso de test.
3. Escribir **tests inteligentes**. No es necesario escribir tests para los getters y setters. En cambio, hay que enfocarse en el comportamiento central de cada componente.

Consejos

1. Visualizar el testing unitario como una **disciplina** asociada con las actividades de desarrollo de software.
2. Escribir un caso de test para cada **elemento individual de funcionalidad**. Cada afirmación individual en la especificación debe tener al menos un caso de test.
3. Escribir **tests inteligentes**. No es necesario escribir tests para los getters y setters. En cambio, hay que enfocarse en el comportamiento central de cada componente.
4. **Setear un entorno limpio** para cada test. No es bueno que un test falle porque un test previo dejó datos inconsistentes o inadecuados.

Consejos

1. Visualizar el testing unitario como una **disciplina** asociada con las actividades de desarrollo de software.
2. Escribir un caso de test para cada **elemento individual de funcionalidad**. Cada afirmación individual en la especificación debe tener al menos un caso de test.
3. Escribir **tests inteligentes**. No es necesario escribir tests para los getters y setters. En cambio, hay que enfocarse en el comportamiento central de cada componente.
4. **Setear un entorno limpio** para cada test. No es bueno que un test falle porque un test previo dejó datos inconsistentes o inadecuados.
5. Usar **objetos falsos** para no actuar sobre datos importantes.

Consejos

6. **Agregar nuevos tests** cuando se modifica la funcionalidad. Es importante mantener los tests actualizados!

Consejos

6. **Agregar nuevos tests** cuando se modifica la funcionalidad. Es importante mantener los tests actualizados!
7. Escribir tests **antes de corregir un bug**. Es una forma de asegurarse de que el bug se corrigió, y también permite detectar si el bug reaparece.

Consejos

6. **Agregar nuevos tests** cuando se modifica la funcionalidad. Es importante mantener los tests actualizados!
7. Escribir tests **antes de corregir un bug**. Es una forma de asegurarse de que el bug se corrigió, y también permite detectar si el bug reaparece.
8. Usar tests unitarios para asegurar que la **performance** del sistema no empeora.

Consejos

6. **Agregar nuevos tests** cuando se modifica la funcionalidad. Es importante mantener los tests actualizados!
7. Escribir tests **antes de corregir un bug**. Es una forma de asegurarse de que el bug se corrigió, y también permite detectar si el bug reaparece.
8. Usar tests unitarios para asegurar que la **performance** del sistema no empeora.
9. Ejecutar los tests **continuamente**. La situación ideal es que los tests se ejecuten automáticamente, y se envíen avisos a los desarrolladores si se encuentran problemas.

Consejos

6. **Agregar nuevos tests** cuando se modifica la funcionalidad. Es importante mantener los tests actualizados!
7. Escribir tests **antes de corregir un bug**. Es una forma de asegurarse de que el bug se corrigió, y también permite detectar si el bug reaparece.
8. Usar tests unitarios para asegurar que la **performance** del sistema no empeora.
9. Ejecutar los tests **continuamente**. La situación ideal es que los tests se ejecuten automáticamente, y se envíen avisos a los desarrolladores si se encuentran problemas.
10. **Divertirse** testeando!

Consejos

When I first encountered unit testing, I was sceptical and thought it was just extra work. But I gave it a chance, because smart people who I trusted told me that it's very useful. Unit testing puts your brain into a state which is very different from coding state. It is challenging to think about what is a simple and correct set of tests for this given component.

Once you start writing tests, you'd wonder how you ever got by without them. To make tests even more fun, you can incorporate pair programming. Whether you get together with fellow developers to write tests or write tests for each other's code, fun is guaranteed. At the end of the day, you will be comfortable knowing your system really works because your tests pass.

– Alex Iskold, 2008.